

A toolkit for building cross-platform migratory software applications

Lakshitha Rajapakse

Dept. of CSE

University of Moratuwa

Katubedda, Sri Lanka

lakshitha.15@cse.mrt.ac.lk

Shehan Karunarathna

Dept. of CSE

University of Moratuwa

Katubedda, Sri Lanka

shehanhasanga.15@cse.mrt.ac.lk

Rajitha Kumara

Dept. of CSE

University of Moratuwa

Katubedda, Sri Lanka

tmrkumara.15@cse.mrt.ac.lk

Kutula Gunasekera

Dept. of CSE

University of Moratuwa

Katubedda, Sri Lanka

kutula@cse.mrt.ac.lk

Abstract—People today tend to use multiple smart devices such as smartphones, tablet PCs, laptops etc. While these devices are typically used for different tasks, and in different situations, sometimes it is necessary to shift our work from one device to another due to reasons such as personal mobility or limitations of the current device. Currently there is no standard mechanism to directly move an application and its working state between heterogeneous smart devices. This paper proposes a toolkit to develop cross-platform migratory software applications as a solution to this problem. We describe our design, investigate three current technologies for implementation of the same and report our initial evaluations.

Index Terms—Application Migration, Code Mobility, Platform Independent

I. INTRODUCTION

Individuals today own and use multiple smart devices. For example, a person may, within a single day, use a smartphone, a tablet PC, a laptop computer, a work computer, and a home theatre PC. While these devices may be used in different situations and for different tasks, sometimes it is necessary to shift a task from one device to another.

Let us consider, Cyril, an office worker who is working on an important report that needs to be completed by midnight. He is working on it at the office, but needs to leave at 4.30pm to catch the train home. He then continues to work on the report during the train commute using his tablet PC. Half way through the commute, his tablet PC runs out of battery, so the work gets shifted to his smartphone, not an ideal device to work on, but the only option available. Then, at home he switches to his home laptop and is able to finish the report before midnight.

What is of interest here is how “work” gets shifted from one device to another. An obvious choice today is to use a cloud-based application (e.g. Google Docs, Office365) or a storage service (e.g. Dropbox, OneDrive). However, this option is not possible if the work is highly confidential, the work-in-progress is too large (e.g. media files) to be sent back and forth over the Internet, or simply if Internet connectivity is not available. The standard mechanism then would be to save the work, transfer the file to the other device (e.g. using a USB pen drive), open it on the destination device and resume work. Sometimes, a peer-to-peer storage synchronizing service (e.g. Syncthing.net) could be used. Needless to say, this is both

inconvenient and time consuming. What if it was possible to move the work from one device to another at the press of a button, and immediately resume work from the second device? In this paper, we propose such a solution - an approach to build software applications that can migrate across devices while preserving their state.

An application can be separated into two parts, process and user-interface (UI). If someone wants to send an application from one device to another, process and UI have to be migrated separately. Here process means current executing or suspended states of the application. In process migration, this process continues its execution while moving to different machines across network. Process migration can be divided into two parts such as preemptive process migration and non-preemptive process migration. In non-preemptive process migration, migration takes place before execution of the process starts. In preemptive process migration, process is preempted, migrated and continued processing in a different execution environment. Process migration has main steps such as freezing the process on its source node and restarting it on destination node, moving process address space and handling communication between cooperating processes that are depend on operating platforms. So that, developers have to handle inter-process communication, shared memory management and I/O redirection. When moving across different machines, processes have to change their process ids and manage communication between them. Process is bounded with address space of the current machine. So there must be a way to redirect access to these file resources even after the process is migrated. In the other hand, there is no simple way to retrieve all administrative information about a process into a simple sequence of replicated operations as well. UI means all the graphical content of the application. It is bounded with the execution process that is mentioned above. It is based on Human Computer Interaction (HCI) concepts to capture user interactions in an efficient way. In UI migration there must be a common rendering engine for all different platforms. Otherwise there must be a mechanism to match native rendering elements of different platforms.

There is no any existing mechanism to directly move an application and its working state to another device. This paper introduces an approach to move an application and its work-in-

progress (i.e. state) to another trusted device. Desktop devices running Windows and Linux and smart mobile devices running Android are considered at present.

The rest of the paper is structured as follows. In Section II we provide a survey of related work, and introduce our proposed solution in Section III. Section IV describes our prototype implementations which investigate different current technologies. Section V provides a preliminary performance evaluation and the paper is concluded in Section VI.

II. RELATED WORK

The easy method of transferring “work” from one device to another is using a cloud based system. There are several systems which provide those kinds of services (e.g. Google Docs, Google Drive). Though it is a convenient way, user must have an Internet connection to proceed. Moreover, there are security concerns. If users can achieve application migration over their devices, it would enhance the usability and efficiency.

Application migration is the process of moving an application from one machine to another. For that, source code and data of the application must be portable. It is related to the concept called “Code Mobility”. Code Mobility means transferring code segments or object from one machine to another. Distributed operating systems use code mobility to achieve process migration. These processes are operating system (OS) processes which are invisible to users and introduced at the OS level in order to achieve load balancing. Object migration is also a part of process migration. States and data related to source code should be sent to the destination node. In OS level object migration, objects are moved among address spaces. As an example, Emerald [1] which is a object-based language and system, provides flexible object migration in distributed programming. All these features are hidden to the user and limited to the OS level. But with the development of this technology, better forms of code mobility are developed. They are called “Mobile Code Systems”. In mobile code system, four main design paradigms are introduced.

- Client-Server
- Remote Evaluation
- Code on Demand
- Mobile Agent

When considering these design paradigms, Mobile Agent is the most related paradigm to application migration. Mobile agent is an application which can migrate from one node to another autonomously with the current state and data in a distributed system. Compared to other paradigms mobile agent paradigm reduces network traffic. Mobile agent platforms handle the execution of mobile agents. These platforms consist of two separate parts. Mobile agent which is the software code that can be moved over devices and runtime environments which acts as the server that approves the execution, migration and communication [2]. There are several mobile agent platforms that support application mobility. Most of them are Java based platforms such as Aglets [3], Mole [4], JADE [5], DAgents [6] while some of them are based on JavaScript such as JavaScript middleware [7]. Aglets, Mole

and JADE are developed to work only with Java mobile agents. Communication between nodes, the ability to migrate states and executions, the efficiency of migration, autonomy of the agent and usability are the main factors that are researchers are concerned about.

Among those mobile agent platforms, JADE is more popular and open source agent development platform. It has a runtime environment called container to execute agents. JADE can support mobility of code and preserve execution state also. So these agents can distribute computational load without impact on application [8]. It can handle multiple agents concurrently because it allocates one thread per agent. But it only supports for Java based implementations and currently there is no complete solution for Android devices also. It is not a user oriented application and it only supports UI migrations in desktop devices.

JavaScript middleware can be used for broad deployment of agent based solutions across different platforms. It has mainly three components which are Registry server, Middleware and Mobile agents. Registry server stands as a communication mediator between middleware and mobile agents. Middleware is capable of executing mobile agents. Mobile agents can be implemented using JavaScript. Mobile agents can migrate through network and resume with last state using object serialization. But limitation is that it mainly focuses on process migration and it does not focus on UI migrations. It can only support for single agent execution. It is not a user oriented application also.

Aglets is also a Java based implementation. Specially, Aglets mobile agents can autonomously move from one host to another. As it uses Java, it can support platform independency, secure execution, dynamic class loading, multi-threaded programming and object serialization [9]. Additionally it can support resource management, save ownership of references, preserve and resume execution state also. It has proxy component to access control of mobile agents. But users cannot generalize a most suitable configuration as in JADE. It takes more connection establishment time than JADE. It also has vulnerability of overload of the system.

SAMD [10] is a platform level collaboration framework. It can dynamically send application portions to remote devices and execute them without any pre-installation process. To achieve this, developers have to design their application as a collection of controllers and developers have to decide local and remotable controllers using flags in implementations process. It only supports for Android devices and users have to replace their current Android OS with SAMD enabled Android OS.

By examining all those related works, we have identified that these solutions have their own capabilities and limitations. The limitation with cloud apps is that they cannot work without internet. They can have security problems and they cannot get native device capabilities. In mobile agents platforms, they mainly target on process migration while preserving current states. So they have some limitations on UI migrations. Specially, they are not user oriented and do

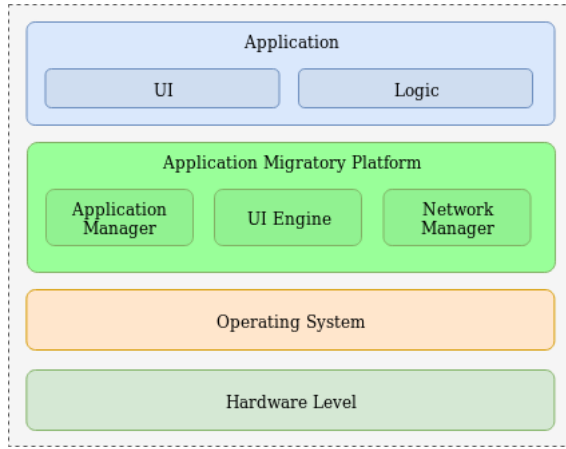


Fig. 1. High-level software stack

not fully support for platform independency. So it is effective to have user oriented application migration platform that can migrate applications while preserving current states across different platforms.

III. APPLICATION MIGRATORY PLATFORM

The *Application Migratory Platform* must support both heterogeneous devices and operating systems such as Android, Windows and Linux. Therefore, separate platform implementations are needed to support these operating systems. Fig. 1 shows the high-level software stack of the proposed *Application Migratory Platform*. The *Application* layer is a software package which can migrate from one device to another. *Applications* are built on top of the *Application Migratory Platform* which is responsible for preserving application's state, migration and abstracting OS-specific details.

The *Application Migratory Platform* consists of three major components such as *Application Manager*, *UI Engine* and *Network Manager*.

A. Platform Components

a) *Application Manager*: *Application Manager* is responsible for loading the migratory application's executable files to platform address space. After loading executable files, it starts a separate process (daemon process) for the migratory application. Migratory application's state is loaded at the application startup. *Application Manager* connects the operating system and the migratory application by making abstract layer between them. When the migratory application is needed to be sent, its state should be preserved and handed over to *Network Manager*.

b) *UI Engine*: This component controls user interface rendering in migratory applications. User interfaces are developed as web based (HTML/JS) components. Therefore, UI can be rendered on webview and Chrome view in Android devices and desktop devices respectively. After initializing the UI, it will be connected with daemon process which is established by *Application Manager*. Then, user can interact with the migratory application. User feels the application as a native

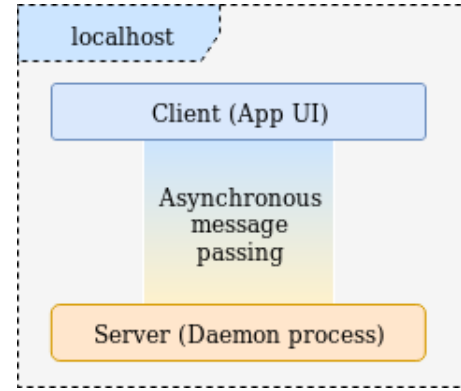


Fig. 2. Migratory application architecture

application on particular operating system. In Android OS, a webview embedded “Activity” is assigned for a migratory application and in desktop, it will be a Chrome view embedded window.

c) *Network Manager*: *Network Manager* handles inter device network communication. The main objective of the *Network Manager* is to send and receive migratory applications. Migratory application can be sent either through peer-to-peer connection or over the Internet as a package (migratory application and state). When network manager receives a package, it will be handed over to the *Application Manager*.

B. Migratory Application Architecture

Client-server architecture is used to mitigate interference of the *Application Migratory Platform* into migratory application. Daemon process acts as the server and application UI acts as client. These are initiated by *Application Manager* and *UI Engine* respectively. Since daemon process and the UI run separately, there must be a medium to communicate with each other according to Fig. 2. Therefore, bidirectional communication channel will be established after initiating application UI.

C. Workflow

Fig. 3 represents the workflow of the *Application Migratory Platform*. After initiating the platform and triggered to loads a particular application, *Application Manager* takes over the control and load the application. Then it fetches states and connects with the *UI Engine*. Then user can interact with

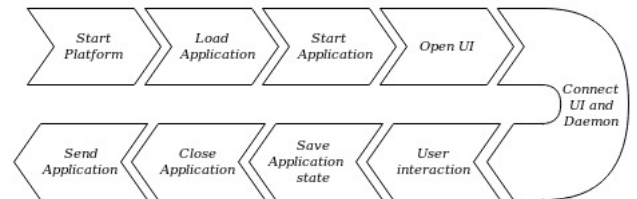


Fig. 3. Work flow

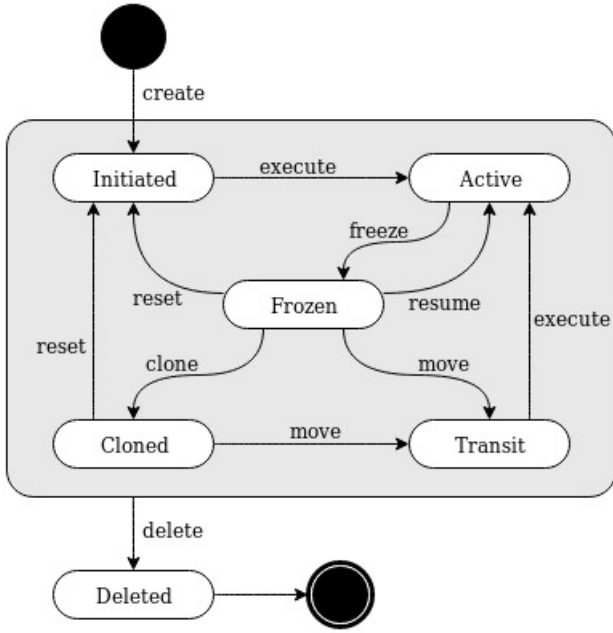


Fig. 4. Migratory application life cycle

the application. Whenever user wants to pause the work and send the application, command must be given to the platform. *Application Manager* saves the current states of the application and hands over the application to the *Network Manager* with its source code. *Network Manager* is responsible for sending the application.

D. Migratory Application Life Cycle

A Migratory application has six states in its life cycle. Fig. 4 represents the state diagram of application. When the application is *Initiated*, platform generates a new “state file” for the application. After triggering “execute”, application's life cycle state is turned into *Active*. When application is needed to be sent, first the states must be preserved. While doing it, the application is sent to *Frozen* state. User can either clone the application and send or just send without preserving the application on the device. When the application is moved to another device that device recognizes application's state as *Transit*. Application can be put in to *Active* state by executing. All these state changes are done by *Application Manager*.

IV. PROTOTYPE IMPLEMENTATIONS

Considering the requirements of *Application Migratory Platform*, three possible technologies were identified. They are described next.

A. Java based approach

In Android operating system, Android runtime (ART) acts as the runtime environment. ART translates application's bytecode into native instructions. ART can execute Dex bytecode specification [11].

When considering desktop operating systems, Java applications can be easily run using Java Runtime Environment (JRE). JRE accepts Java Archive (jar) file format. It includes Java Class files and other metadata. Since ART and JRE accept two different formats, in order to support cross platform migration there should be two different platforms to handle .jar and .dex files in the proposed Java based approach.

Application Migratory Platform for Android OS can be installed on the device using an Android Package Kit (APK). Platform is capable of loading a .dex file dynamically and load the classes in that .dex file. Application which is running on that platform comes within a .dex file. Bytecode of daemon process is bundled in the .dex file. By the time .dex file is loaded, the platform has no idea about the classes or functions in the .dex file. Therefore, a common Java interface is introduced to handle this problem. That interface contains main functions that can be called to the migratory application. Implementation of those functions is unique to the migratory application and it is implemented in the .dex file. Platform loads the .dex file and creates an object referring to the known interface and calls functions in the application through that object.

Web based technologies are used to get the same user interface for both Android and desktop devices. In order to achieve dynamic UI loading, platform load UI from local file system. UI is developed with React.js because it can manage states of the rendering components and dynamically change components without page refreshing. Those states in the UI can be changed using a daemon process that is implemented using Java web sockets.

B. Node.js based approach

In the proposed Node.js based *Application Migratory Platform*, Node.js is the main runtime. On the desktop environment, Node.js can be installed as an application in order to execute JavaScript applications. But in Android it is unable to use Node.js as an application. Therefore, Node.js source code was updated for Android (only for arm64-v8a and armeabi-v7a instruction set) and binary file (shared library) was built as the output. That binary file is embedded into *Application Migratory Platform* which is an Android application. Most Node.js features are supported on Android OS since Android is a modified version of the Linux kernel. After initiating Node.js process, it will run separately from the *Application Migratory Platform*. But there must be a connection between Node.js process and *Application Migratory Platform* to perform UI related tasks. Client-server architecture is used within the same device to combine UI and Node.js process on Android. On Android, Android webview will be used as the UI Engine.

Electron.js [12] is used to implement *Application Migratory Platform* on the desktop devices. It provides facility to create native applications on Windows, Linux and macOS using Node.js and Chrome view. Client-server architecture is used to fill the communication gap between Node.js processes and UI. In order to preserve Node.js process related states, Node.js VM module is used in both desktop and Android implementations.

But there are some limitations when preserving objects. As an example, some process related objects might depend on hardware. Moving these objects to another hardware might cause failures and misbehaviors. Therefore, the solution only allows to preserve states of hardware independent objects (i.e. database connection objects).

To achieve UI related state preserving, UI is implemented using Flux architecture [13] which introduced by Facebook Inc. The architecture comes up with the concept of uni-directional dataflow to manage complex single page web applications. In the solution, migratory application UI is based on combination of HTML and JavaScript. Therefore, React.js [14] is the proper solution to implement UI since it provides stateful components. To fulfill Flux architecture, Redux [15] is embedded in to React.js application. Final output of the migratory application is a hybrid application running on top of the proposed *Application Migratory Platform*.

C. V8 & C++ based approach

V8 engine is used by Google Chrome web browser, Chromium web browser, Microsoft Edge web browser and Node.js runtime as JavaScript compiler. It can be plugged into any C++ application. When executing JavaScript on V8, it will create isolated environment with its own heap. That isolated environment acts as a VM instance [16]. If migratory application can be run on lightweight VM, it can be moved to another device and continue the work by loading the VM on that device. In order to facilitate that feature, VM should not depend on the platform. V8 engine fulfills this requirement. V8 provides a feature called custom startup snapshot [17]. Mobility of the migratory application can be achieved through this feature. Snapshots can be created from a specific point of the application by serializing heap states into binary blob file (*snapshot_blob.bin*). That snapshot contains V8's heap at the time snapshot was created. In high-level, snapshot can be moved to another device and can be executed using V8 instance on the new device. In enterprise applications, this V8 snapshot normally used to speedup startup time [18] [19] [20]. V8 deserialize received custom startup snapshot into the heap in new device. Then V8 starts the application. Since proposed platform moves heap of the application, it seems V8 can provide strong mobility. But it actually reinitialize the application. Therefore, this method provides weak mobility. V8 has a concept called context [21]. Context manage global variable scope within V8 instance. As an example in web browser user can open multiple tabs without having conflicts with each other. Proposed platform is also capable of handling multiple applications at once.

V8 source code was built for ARM64 and ARM architectures and tested on Android (with *arm64-v8a* and *armeabi-v7a* instruction set on ARM Cortex-A53 processor) by embedding to Android application. A custom startup snapshot was successfully generated for a JavaScript script. And that snapshot only supports for device's Application Binary Interface (ABI) type. As an example, snapshots that are created on *arm64-v8a* ABI only support for that same ABI. V8 provides tool

TABLE I
MIGRATION APPLICATION STARTUP & PACKAGING APPROXIMATE TIMES

Device	App startup time (s)		Unpackaging time (s)	Packaging time (s)
	Node.js	UI		
Android 6.0 (Xiaomi Redmi Note 4X)	2.5	4.1	28	6
Android 7.0 (Huawei P9-lite)	3.6	6.9	80	10
Windows 10 (Lenovo Intel i3)	0.5	4.2	5	2
Ubuntu 18.04 (Lenovo Intel i3)	0.4	1.8	2	9

called *mksnapshot* [22] to build snapshot blob file from JavaScript file. But that can only be executed on desktop machine (when cross-compiled [23]) for any CPU architecture to build snapshot blob file. This was a major barrier when going through the V8 based approach. Next critical issue was UI migration with V8 engine. V8 based approach is highly biased towards process offloading. Because V8 is not aware about UI related stuff such as DOM manipulation. Therefore, standalone V8 cannot manage HTML document. To manage UI, DOM wrapper should be embedded for V8 [24] and rendering engine such as Blink [25]. Then proposed migratory platform becomes customized web browser.

Two significant challenges were encountered when implementing the platform using this approach: the V8 version should be compatible with the snapshot; and both sending and receiving devices should have the same CPU architecture. Moreover, V8 cannot get the native support in standalone mode. As such, the V8 & C++ based approach was identified as much less feasible than the other two approaches and discontinued.

V. PERFORMANCE EVALUATION

The performance of Node.js approach was evaluated using a demo application. Evaluation criteria were startup time, memory usage and packaging time (i.e. time taken for bundling the demo application). The demo application has a size of 3.2MB and it was packaged as a ZIP file before being transferred to

TABLE II
APPROXIMATE MEMORY USAGE

Device	Memory usage (MB)		
	Platform only	Platform + Node.js	Platform + Node.js + UI
Android 6.0 (Xiaomi Redmi Note 4X)	37	45 (max:47)	112 (max:138)
Android 7.0 (Huawei P9-lite)	55	65 (max:82)	96 (max:138)
Windows 10 (Lenovo Intel i3)	23	172 (max:185)	215 (max:235)
Ubuntu 18.04 (Lenovo Intel i3)	34	180 (max:180)	230 (max:230)

another device. All sharing processes are done with the same network conditions. This evaluation process was conducted on Android, Linux and Windows platforms by taking average measurements from 20 iterations. Time was calculated using Node.js `console.time()`.

A. Start up time

App startup time is evaluated using Node.js startup time and UI loading time. According to Table I, desktop devices has less startup time compared to Android devices. Android devices spend some time to start Node.js process but After initiating, it can load UI in webview much faster. Although desktop devices can start Node.js process quickly, it takes much time to load UI.

B. Memory usage

Memory usage data in Table II is measured for platform, Node.js process and UI. Specially, Android devices allocate less memory resources than desktop devices. In desktop devices Node.js process get much memory space. But Android devices consume much space for webview.

C. Packaging time

Demo application should be packaged into a ZIP before sending and unpackaged after receiving. Usually unpackaging process get much time. But Linux complete unpackaging with in short period of time.

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed a novel *Application Migratory Platform* to achieve direct application migration between heterogeneous smart devices. It is user-oriented and does not expect low level configuration setup by users as in mobile agent toolkits. At present, we have demonstrated application migration between Android devices as well as between Android and desktop devices (running Windows and Linux). Three current technologies were considered for implementing the platform and prototypes are currently being developed using Java and Node.js.

Evaluation was done using the Node.js implementation. With this approach it is feasible to work platform independently while preserving application state. When compared to native applications the startup time is high, but after initiating, performance is acceptable. If the application contains a higher number of dependencies (i.e. `npm_packages`) then packaging time is higher.

We are currently working on refining the current prototypical platforms, and conducting further evaluations to compare their performance. As future work, we intend to develop several sample applications on top of the *Application Migratory Platform*. Handling differences in sensor availability across devices, and providing per-application permissions instead of platform-wide permissions are also within our roadmap.

REFERENCES

- [1] J.Eric, L.Henry, H.Norman, and B.Andrew, "Fine-grained mobility in the emerald system," *ACM Transactions on Computer Systems*, vol. 6, no. 1, pp. 109–133, February 1998.
- [2] R. S. Chowhan and R. Purohit, "Study of mobile agent server architectures for homogeneous and heterogeneous distributed systems," *International Journal of Computer Applications*, vol. 156, no. 4, December 2016.
- [3] D. Lange and M. Oshima, *Programming and Deploying Java Mobile Agents Aglets*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1998.
- [4] J. Baumann, F. Hohl, K. Rothermel, M. Strasser, and W. Theilmann, "Mole: a mobile agent system," *SoftwarePractice & Experience - Special issue: Mobile agent systems*, vol. 32, no. 6, pp. 575 – 603, May 2002.
- [5] Jade site-java agent development framework. Telecom Italia SpA. [Online]. Available: <http://jade.tilab.com/>
- [6] R. Gray, G. Cybenko, D. Kotz, R. Peterso, and D. Rus, "Dagents: applications and performance of a mobileagent system," *Software-Practice and Experience*, vol. 32, no. 6, p. 543573, 2002.
- [7] C. A. S. Villafuerte and N. Costa, "Framework for supporting javascript-based mobile agents," in *Proceedings of the International Conference on Information Technology & Systems*, Manab, 2018.
- [8] F.Bellifemine, G.Caire, A.Poggi, and G.Rimassa, "Jade: A software framework for developing multi-agent applications. lessons learned," *Information and Software Technology*, vol. 1, no. 50, pp. 10–21, 2008.
- [9] M. Yadav, P. Sethi, D. Juneja, and N. Chauhan, "Development of mobile agents with aglets," *International Journal of Innovations & Advancement in Computer Science*, vol. 4, May 2015.
- [10] J. Lee, H. Lee, B. Seo, Y. C. Lee, H. Han, and S. Kang, "Samd: Fine-grained application sharing for mobile collaboration," in *IEEE International Conference on Pervasive Computing and Communications*, Nagoya, Japan, 2018.
- [11] Android runtime (art) and dalvik-android open source project. Android Open Source Project. [Online]. Available: <https://source.android.com/devices/tech/dalvik/>
- [12] Electron.build cross platform desktop apps with javascript, html, and css. Electronjs.org. [Online]. Available: <https://electronjs.org/>
- [13] Flux.application architecture for building user interfaces. Facebook Inc. [Online]. Available: <http://facebook.github.io/flux/>
- [14] React.a javascript library for building user interfaces. reactjs.org. [Online]. Available: <https://reactjs.org/>
- [15] Redux.a predictable state container for js apps. Redux.js.org. [Online]. Available: <https://redux.js.org/>
- [16] Getting started with embedding v8 v8. V8.dev. [Online]. Available: <https://v8.dev/docs/embed/>
- [17] Custom startup snapshots. V8.dev. [Online]. Available: <https://v8.dev/blog/custom-startup-snapshots/>
- [18] How to build nativescript apps that start up fast. NativeScript.org. [Online]. Available: <https://docs.nativescript.org/performance-optimizations/startup-times#step-3/>
- [19] Protect javascript source code with v8 snapshot. [Online]. Available: <https://github.com/nwjs/nw.js/wiki/protect-javascript-source-code-with-v8-snapshot/>
- [20] V8snapshot - qt wiki. Wiki.qt.io. [Online]. Available: <https://wiki.qt.io/V8Snapshot/>
- [21] Design of v8 bindings. Chromium.googlesource.com. [Online]. Available: https://chromium.googlesource.com/chromium/src.git/+62.0.3178.1/third_party/WebKit/Source/bindings/core/v8/V8BindingDesign.md#Context/
- [22] V8 torque user manual. V8.dev. [Online]. Available: <https://v8.dev/docs/torque#how-torque-generates-code/>
- [23] Cross-compiling and debugging for arm/android. V8.dev. [Online]. Available: <https://v8.dev/docs/cross-compile-arm/>
- [24] Design of v8 bindings. Chromium.googlesource.com. [Online]. Available: https://chromium.googlesource.com/chromium/src/+master/third_party/blink/renderer/bindings/core/v8/V8BindingDesign.md#DOM-wrappers-and-worlds/
- [25] Blink-the chromium projects. Chromium.org. [Online]. Available: <https://chromium.org/blink/>